

From Diffusion to Flow: Principles and Scientific Applications of Modern Generative Models

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

November 14, 2025

Introduction

This lecture is a compact form of the course “Large Foundation Models: Mathematics, Algorithms, and Applications” on the generative modeling and AI for science.

The course materials are available by

- Slides: https://github.com/cam1681/CS25_Large_Foundation_model
- Videos: <https://bimsa.net/activity/LarFouModMatAlgandApp/>

What I cannot create, I do not understand.

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Preliminary: Generate samples from given distribution

Given a distribution $p(x)$, how to sample from the distribution?

- Uniform distribution & Gaussian distribution: easy sampling
- Mixture Gaussian: determine which Gaussian to sample & sample from that Gaussian
- A general sample function?
 - Langevin Markov Chain Monte Carlo sampling (Langevin MCMC)
 - Stein Variational Gradient Descent (SVGD)
 - ...

Score function is all you need?

A typical process of generating new samples from modeling the data distribution

- ① To find the probability function $p(x)$, model $p(x; \theta)$.
- ② Normalization distribution $p(x; \theta) = \frac{1}{C(\theta)} e^{-E(x; \theta)}$ with $C(\theta) \equiv \int_{\mathcal{X}} e^{-E(x; \theta)} dx$.
- ③ Generating samples from above sampling methods.

Recall the Langevin Markov Chain Monte Carlo sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} \nabla_x \log p(x_{t-1}; \theta) + \sqrt{\Delta t} \epsilon, \quad \epsilon \sim N(0, 1). \quad (1)$$

Here $\nabla_x \log p(x_{t-1}; \theta) = -\nabla_x E(x; \theta)$ is free of the normalization $C(\theta)$.

Define the score function

$$S(x; \theta) \equiv \nabla \log p(x; \theta). \quad (2)$$

Molecular Dynamics (MD) and Langevin MCMC

Continuous form of the Langevin MCMC

- $\Delta t \rightarrow 0$

$$dx = \frac{1}{2} \nabla_x \log p(x) dt + dB_t, \quad (3)$$

where dB_t is a Brownian motion.

- Given $p(x) = \frac{1}{C} e^{-E}$, we have

$$dx = -\frac{1}{2} \nabla_x E dt + dB_t \quad (4)$$

Langevin MCMC is overdamped Langevin dynamics

- Langevin dynamics (MD)

$$\ddot{x} = -\nabla_x E - \gamma \dot{x} + dB_t, \quad (5)$$

- Given $\gamma \dot{x} \gg \ddot{x}$, we have the overdamped Langevin equation

$$dx = -\frac{1}{\gamma} \nabla_x E + \frac{1}{\gamma} dB_t.$$

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Matching the score by neural networks

Recall the definition

$$S \equiv \nabla_x \log p(x), \quad (7)$$

The key is to matching the score by a neural network

$$J_{ESM} = \frac{1}{2} \mathbb{E}_p \left[\left\| s(x; \theta) - \frac{\partial \log p(x)}{\partial x} \right\|^2 \right], \quad (8)$$

where J_{ESM} is called Explicit Score Matching loss. However, it is not tractable to compute $\frac{\partial \log p(x)}{\partial x}$ from data.

Where should we go?

Implicit Score Matching (ISM) loss

Implicit Score Matching loss (Aapo, 2005) was developed to make a tractable score matching

$$J_{ISM} \equiv \mathbb{E}_p \left[\frac{1}{2} \|s(x; \theta)\|^2 + \nabla \cdot s(x; \theta) \right] = J_{ESM} - C. \quad (9)$$

Remark

The ISM loss is not very stable to optimize, with two reasons

- 1 Expectation is performed on the whole distribution;
- 2 The loss is negative decreasing to $-C$ with a great quantity, e.g. $-1e5$.

Denoising Score Matching

Denoising Score Matching (DSM) loss (Vincent, 2011) solves the optimization stability problem:

$$J_{DSM} = \frac{1}{2} \mathbb{E}_{p(x, \tilde{x})} \left[\|s(\tilde{x}; \theta) - \nabla_{\tilde{x}} \log p(\tilde{x}|x)\|^2 \right]. \quad (10)$$

Key insight: $J_{DSM} = J_{ESM} + C$ where C is a constant, making it tractable without computing the true score.

Challenge: When $\tilde{x} \rightarrow x$ (low noise), $J_{DSM} \rightarrow \infty$. This motivates the need for **multi-scale noise schedules** in diffusion models.

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Revisit the adding noise from x to $\tilde{x}$

Gaussian transition kernel for adding noise from x to $\tilde{x}$

$$\tilde{x} = \alpha x + \sigma \epsilon, \quad (11)$$

which equivalent to the Gaussian transition kernel

$$p(\tilde{x}|x) = N(\tilde{x}; \alpha x, \sigma^2), \quad (12)$$

where $\epsilon \sim N(0, 1)$ and

$$N(\tilde{x}; \alpha x, \sigma^2) = C e^{-\frac{\|\tilde{x} - \alpha x\|^2}{2\sigma^2}}. \quad (13)$$

And

$$\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}. \quad (14)$$

Questions arise: how can we get x given $\tilde{x}$?

Tweedie's Formula

Tweedie's Formula (Bayes statistics) states (Robbins, 1956)

Theorem

Given random variables x, y , $y \sim N(x, \sigma^2 I)$, i.e., $p(y|x) = N(x, \sigma^2)$, the expectation of x could be given by

$$\mathbb{E}[x|y] = y + \sigma^2 \nabla \log p(y). \quad (15)$$

Let

$$x \leftarrow \alpha x, \quad y \leftarrow \tilde{x}, \quad (16)$$

we have

$$s \equiv \nabla_{\tilde{x}} \log p(\tilde{x}) = -\frac{\tilde{x} - \alpha \mathbb{E}[x|\tilde{x}]}{\sigma^2} = -\frac{1}{\sigma} \mathbb{E}\left[\frac{\tilde{x} - \alpha x}{\sigma} | \tilde{x}\right] = -\frac{\mathbb{E}[\epsilon|\tilde{x}]}{\sigma}, \quad (17)$$

where we have used the fact $\epsilon = \frac{\tilde{x} - \alpha x}{\sigma}$.

Another insight: from the definition of the score S

Recall that adding noise by $\tilde{x} = \alpha x + \sigma \epsilon$,

$$p(\tilde{x}|x) = Ce^{-\frac{\|\tilde{x}-\alpha x\|^2}{2\sigma^2}}. \quad (18)$$

Hence

$$s(\tilde{x}) = \nabla_{\tilde{x}} \log p(\tilde{x}) = \frac{\nabla_{\tilde{x}} p(\tilde{x})}{p(\tilde{x})} = \frac{\int_x \nabla_{\tilde{x}} p(\tilde{x}|x) p(x) dx}{p(\tilde{x})}, \quad (19)$$

From Eq. (18), we have

$$s(\tilde{x}) = \frac{\int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x) p(\tilde{x}|x) p(x) dx}{p(\tilde{x})} = \int_x \nabla_{\tilde{x}} \log p(\tilde{x}|x) p(x|\tilde{x}) dx, \quad (20)$$

leading to

$$s(\tilde{x}) = \mathbb{E}_x[\nabla_{\tilde{x}} \log p(\tilde{x}|x) | \tilde{x}] = \mathbb{E}_x[-\frac{\tilde{x} - \alpha x}{\sigma^2} | \tilde{x}] = -\frac{\tilde{x} - \alpha \mathbb{E}[x | \tilde{x}]}{\sigma^2}.$$

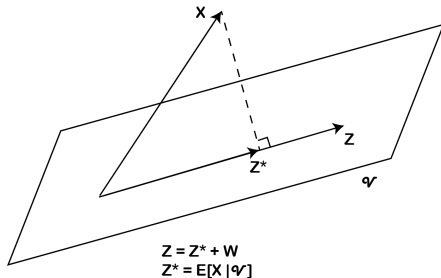
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for any $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X | \mathcal{V})$ is the optimal solution of the least square problem

$$\operatorname{argmin}_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = (\int Z^2 dP)^{\frac{1}{2}}$.



Revisit several models

① Score model

$$Loss_{score} = \|s_{\theta}(\tilde{x}) - \frac{\epsilon}{\sigma}\|^2; \quad (22)$$

② X prediction model (denoising model, x_0 model)

$$Loss_{x_0} = \|x_{\theta}(\tilde{x}) - x\|^2, \quad (23)$$

with

$$s(\tilde{x}) = -\frac{\tilde{x} - \alpha x_{\theta^*}(\tilde{x})}{\sigma^2} \quad (24)$$

where $x_{\theta}(\tilde{x}) \rightarrow x_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[x|\tilde{x}]$;

③ ϵ prediction model

$$Loss_{\epsilon} = \|\epsilon_{\theta}(\tilde{x}) - \epsilon\|^2, \quad (25)$$

with

$$s(\tilde{x}) = -\frac{\epsilon_{\theta^*}(\tilde{x})}{\sigma}, \quad (26)$$

where $\epsilon_{\theta}(\tilde{x}) \rightarrow \epsilon_{\theta^*}(\tilde{x}) \equiv \mathbb{E}[\epsilon|\tilde{x}]$.

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - **Potential Score Matching**
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Key Identity: Jacobian Switch (chain rule between $\mathbf{x}_0$ and $\mathbf{x}_t$)

The Gaussian transition density is

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \alpha_t \mathbf{x}_0, \sigma_t^2 \mathbf{I}) \propto \exp\left(-\frac{\|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2}{2\sigma_t^2}\right).$$

A basic but crucial identity:

$$\nabla_{\mathbf{x}_t} q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{1}{\alpha_t} \nabla_{\mathbf{x}_0} q(\mathbf{x}_t | \mathbf{x}_0), \quad (\alpha_t > 0).$$

Proof sketch. Differentiating the exponent w.r.t. $\mathbf{x}_t$ and w.r.t. $\mathbf{x}_0$ shows

$\nabla_{\mathbf{x}_t} \|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2 = 2(\mathbf{x}_t - \alpha_t \mathbf{x}_0)$, $\nabla_{\mathbf{x}_0} \|\mathbf{x}_t - \alpha_t \mathbf{x}_0\|^2 = -2\alpha_t(\mathbf{x}_t - \alpha_t \mathbf{x}_0)$, thus the factor $-1/\alpha_t$.

Theorem 1: Score as Conditional Force Expectation

Statement. With $p(\mathbf{x}_0) \propto e^{-E(\mathbf{x}_0)/k_B T}$ and $\mathbf{x}_t = \alpha_t \mathbf{x}_0 + \sigma_t \epsilon$,

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right], \quad \mathbf{F}(\mathbf{x}_0) = -\nabla_{\mathbf{x}_0} E(\mathbf{x}_0).$$

Derivation.

$$\begin{aligned} \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) &= \frac{\nabla_{\mathbf{x}_t} \int p(\mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0}{p(\mathbf{x}_t)} \\ &= \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \nabla_{\mathbf{x}_t} q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0 \end{aligned}$$

Theorem 1: Derivation (Cont)

Proof (Cont).

Using the Jacobian switch identity (*):

$$\begin{aligned} &\stackrel{(*)}{=} \frac{1}{p(\mathbf{x}_t)} \int p(\mathbf{x}_0) \left(-\frac{1}{\alpha_t} \right) \nabla_{\mathbf{x}_0} q(\mathbf{x}_t | \mathbf{x}_0) d\mathbf{x}_0 \\ &= -\frac{1}{\alpha_t} \frac{1}{p(\mathbf{x}_t)} \int q(\mathbf{x}_t | \mathbf{x}_0) \nabla_{\mathbf{x}_0} p(\mathbf{x}_0) d\mathbf{x}_0 \end{aligned}$$

Theorem 1: Derivation (Final)

Proof (Cont).

$$\begin{aligned} &= -\frac{1}{\alpha_t} \int \underbrace{\frac{p(\mathbf{x}_0)q(\mathbf{x}_t | \mathbf{x}_0)}{p(\mathbf{x}_t)}}_{p(\mathbf{x}_0|\mathbf{x}_t)} \nabla_{\mathbf{x}_0} \log p(\mathbf{x}_0) \, d\mathbf{x}_0 \\ &= \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0|\mathbf{x}_t} \left[\frac{-\nabla_{\mathbf{x}_0} E(\mathbf{x}_0)}{k_B T} \right] = \frac{1}{\alpha_t} \mathbb{E}_{\mathbf{x}_0|\mathbf{x}_t} \left[\frac{\mathbf{F}(\mathbf{x}_0)}{k_B T} \right] \end{aligned}$$

Experiments on first 1, 000 frames for LJ data

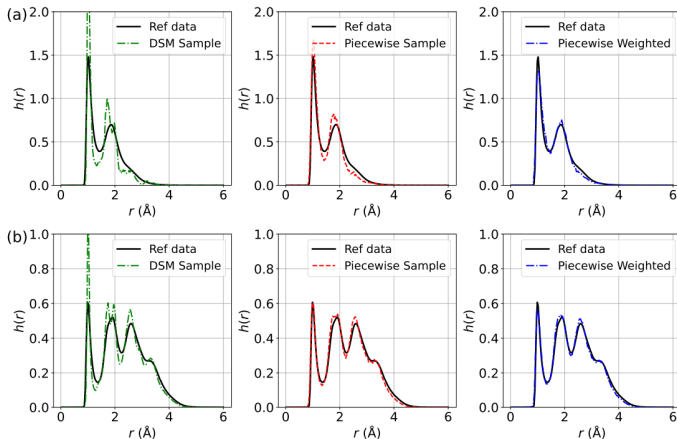


Figure 1: The distribution of interatomic distances $h(r)$ for LJ potential using biased training data with a sample size of 500. (a) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses (from left to right) against reference data for LJ-13; (b) Comparison of $h(r)$ for DSM, Piecewise, and Piecewise Weighted losses for LJ-55.

Experiments on first 5, 000 frames for MD reference trajectories

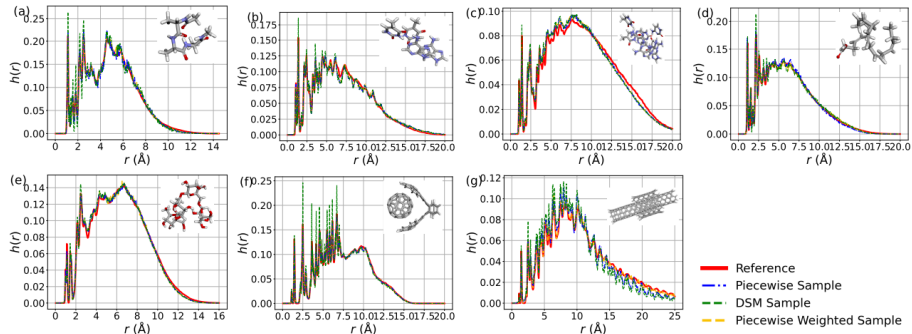


Figure 3: The distribution of interatomic distances ($r(\text{\AA})$) for (a) Ac-Ala3-NHMe, (b) DNA base pair (AT-AT), (c) DNA base pair (AT-AT-CG-CG), (d) Docosahexaenoic acid, (e) Stachyose, (f) Buckyball catcher, (g) Dw nanotube in MD22 dataset. The insets display the ball-and-stick representations of these molecules.

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

From Theory to Practice

Now that we have established the theoretical foundations of score matching and diffusion processes, let's explore how these concepts are implemented in practice through several key approaches:

- SMLD (Score Matching Langevin Dynamics)
- DDPM (Denoising Diffusion Probabilistic Models)
- Continuous formulations and their advantages

Each approach offers different insights into the practical implementation of diffusion-based generative models.

Score Matching Langevin Dynamic (SMLD) (Song, 2019) adding noise in a following manner

$$p(\tilde{x}_i|x) = N(\tilde{x}_i; x, \sigma_i^2) \equiv Ce^{-\frac{\|\tilde{x}_i - x\|^2}{2\sigma_i^2}}, \quad (27)$$

where a geometric sequence $\sigma_1 > \sigma_2 > \dots > \sigma_T \approx 0$ is given to add noise with different level. In a random variable view,

$$\tilde{x}_i = x + \sigma_i \epsilon \quad (28)$$

for different σ_i to get different noised random variable $\tilde{x}_i$.

Training object J_{DSM} is given by

$$J_{DSM} = \frac{1}{2} \mathbb{E}_{\sigma_i} \mathbb{E}_{p(x)} \mathbb{E}_{p(\tilde{x}_i|x)} \left[\left\| s_{\theta}(\tilde{x}_i, \sigma_i) + \frac{\tilde{x}_i - x}{\sigma_i^2} \right\|^2 \right]. \quad (29)$$

Anneled Langevin MCMC for sampling

$$x_t = x_{t-1} + \frac{\Delta t}{2} S_{\theta}(x_{t-1}) + \sqrt{\Delta t} \epsilon, \quad \epsilon \sim N(0, 1).$$

denoising diffusion probabilistic models (DDPM) (Ho, 2020)

- ① Derived from the Evidence Lower BOund (ELBO), **Not** score matching.
- ② First provide adding noise and denoise in the forward and reverse process.

The main procedure

- ① Adding noise

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon; \quad (31)$$

- ② Transition kernel

$$p(x_t|x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \quad (32)$$

where $\alpha_t = \prod_{i=1}^t \sqrt{1 - \beta_i}$ and $\sigma_t^2 = 1 - \alpha_t^2$.

The main procedure (Continued)

- 1 Adding noise

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon; \quad (33)$$

- 2 Transition kernel

$$p(x_t | x_0) = N(x_t; \alpha_t x_0, \sigma_t^2), \text{ i.e., } x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad (34)$$

where $\alpha_t = \prod_{i=1}^t \sqrt{1 - \beta_i}$ and $\sigma_t^2 = 1 - \alpha_t^2$.

- 3 Training Loss

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{p(x)} \mathbb{E}_{p(x_t|x)} \left[\|\epsilon_\theta(x_t, t) - \epsilon\|^2 \right], \quad (35)$$

where we have used the approximation $x_0 = \frac{x_t - \sigma_t \epsilon}{\alpha_t}$.

- 4 Sampling

$$x_{t-1} = \frac{1}{\sqrt{1 - \beta_t}} (x_t + \beta_t s_\theta(x_t, t)) + \sqrt{\beta_t} \epsilon_t, \quad t = T, T-1, \dots, 1. \quad (36)$$

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

The continuous version of SMLD

Recall that

$$x_t = x_0 + \sigma_t \epsilon, \quad (37)$$

we can prove that

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon. \quad (38)$$

Proof.

We can prove the formula by a Recurrence

$$\begin{aligned} x_t &= x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon \\ &= x_{t-2} + \sqrt{\sigma_{t-1}^2 - \sigma_{t-2}^2} \epsilon + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon \\ &= x_{t-2} + \sqrt{\sigma_t^2 - \sigma_{t-2}^2} \epsilon \\ &= \dots = x_0 + \sigma_t \epsilon, \quad \text{where } \sigma_0 \approx 0. \end{aligned} \quad (39)$$

The continuous version of SMLD

From

$$x_t = x_{t-1} + \sqrt{\sigma_t^2 - \sigma_{t-1}^2} \epsilon, \quad (40)$$

we have

$$\begin{aligned} \Delta x_t &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}} \sqrt{\Delta t} \epsilon \\ &= \sqrt{\frac{\sigma_t^2 - \sigma_{t-1}^2}{\Delta t}} \Delta B_t. \end{aligned} \quad (41)$$

Further, let $\Delta t \rightarrow 0$, we have

$$dx = \sqrt{\frac{d\sigma_t^2}{dt}} dB_t. \quad (42)$$

The continuous version of DDPM

Recall that

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon. \quad (43)$$

Similarly

$$x_t \approx \left(1 - \frac{1}{2} \beta_t\right) x_{t-1} + \sqrt{\beta_t} \epsilon, \quad (44)$$

hence, we have

$$\begin{aligned} \Delta x_t &= -\frac{1}{2} \tilde{\beta}_t x_{t-1} \Delta t + \sqrt{\tilde{\beta}_t} \sqrt{\Delta t} \epsilon, \quad \text{where } \tilde{\beta}_t = \frac{\beta_t}{\Delta t} \\ &= -\frac{1}{2} \tilde{\beta}_t x_{t-1} \Delta t + \sqrt{\tilde{\beta}_t} \Delta B_t. \end{aligned} \quad (45)$$

Further, let $\Delta t \rightarrow 0$, we finally obtain

$$dx = -\frac{1}{2} \tilde{\beta}_t x dt + \sqrt{\tilde{\beta}_t} dB_t. \quad (46)$$

Summarizing the form of continuous DDPM and SMLD, we summarize the adding noise process as

$$dx = f(x, t)dt + g(t)dB_t, \quad (47)$$

where the corresponding distribution $p(x, t)$ satisfying the following Fokker-Planck equation

$$\frac{\partial p(x, t)}{\partial t} + \nabla \cdot (f(x, t)p(x, t)) - \frac{1}{2}g^2(t)\Delta p(x, t) = 0. \quad (48)$$

Training DSM object

$$J_{DSM} = \mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{p(x_0)} \mathbb{E}_{p(x_t|x_0)} \left[\|s_\theta(x_t, t) - \nabla_{x_t} \log p(x_t|x_0)\|^2 \right]. \quad (49)$$

Training recipe

The training for VE follows the following process (VP is in a similar way)

- 1 Random choose one data $x \sim p_{data}$;
- 2 Random sample $t \sim U[0, 1]$;
- 3 Random sample a white noise $\epsilon \sim N(0, 1)$;
- 4 Adding noise

$$x_t = \tilde{\alpha}_t x + \tilde{\sigma}_t \epsilon, \quad (50)$$

where the mean value $\tilde{\alpha}_t x = x(0)$ and standard deviation $\tilde{\sigma}_t = \sqrt{\sigma^2(t) - \sigma^2(0)} \approx \sigma(t)$ in VE referring to (??). VP is similar with the mean and standard deviation from its transition kernel (??);

- 5 Compute the Loss by summation the following norm over x , t and ϵ

$$\|s_\theta(x_t, t) - \epsilon / \tilde{\sigma}_t\|.$$

Inference: Reverse denoising process

The reverse denoising process is given by

$$dx = (f(x, t) - g^2 \nabla_x \log p(x, t))dt + g(t)dB_t, \quad (52)$$

or

$$dx = (f(x, t) - \frac{1}{2}g^2 \nabla_x \log p(x, t))dt, \quad (53)$$

or

$$dx = (f(x, t) - \frac{3}{2}g^2 \nabla_x \log p(x, t))dt + \sqrt{2}g(t)dB_t, \quad (54)$$

$$\dots \quad (55)$$

Due to obeying the same Fokker-Planck equation for $p(x, t)$

$$\frac{\partial p(x, t)}{\partial t} + \nabla \cdot (f(x, t)p(x, t)) - \frac{1}{2}g^2(t)\Delta p(x, t) = 0. \quad (56)$$

Inference recipe

The inference for VE follows the following process (VP is in a similar way)

- 1 Random generate a noise at time $t = 1$ as $x(1) \sim N(0, \sigma_{max}^2)$;
- 2 Integrate the reverse sampling equation

$$dx = \left(f(x, t) - g^2(t)s_{\theta}(x, t) \right) dt + g(t)dB_t \quad (57)$$

or

$$dx = \left(f - \frac{1}{2}g^2s_{\theta} \right) dt \quad (58)$$

over the time span $[0, 1]$ to get $x(0)$.

- 3 Corrector process: at each internal value $x(t)$, we can relax the value $x(t)$ by a corrector, which takes the Langevin MCMC process.

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Outline

- ① Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- ② Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- ③ Applications in AI for Science: From Sequence to structure prediction

Probability Path and Coupling

- Let $\{p_t\}_{t \in [0,1]}$ be a prescribed path of marginals on $\mathbb{R}^d$ with p_0 simple and p_1 the data.
- A coupling induces random trajectories $t \mapsto x_t$ with instantaneous velocity $\dot{x}_t = \frac{d}{dt}x_t$ (may be stochastic given x_t).
- Goal: learn a deterministic vector field $v : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ such that the ODE

$$\frac{d}{dt}x_t = v(x_t, t), \quad x_0 \sim p_0,$$

yields marginals $x_t \sim p_t$ for all t .

Remark

We can depict the flow matching and diffusion through the probability transition v.s. the trajectories. And also, we can connect the Neural ODE with the Flow matching.

Strong and Weak Continuity

From the Fokker–Planck equation, for $\frac{dx}{dt} = v(x, t)$, we have

$$\partial_t p_t(x) + \nabla \cdot (p_t(x) v(x, t)) = 0.$$

The corresponding weak form of above strong form is

Weak form (for any test functions $\varphi \in C_c^\infty(\mathbb{R}^d)$)

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) p_t(x) dx = \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot v(x, t) p_t(x) dx,$$

i.e.,

$$\frac{d}{dt} \mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla \varphi(x_t) \cdot v(x, t)].$$

Weak Form from Random Trajectories

Let x_t be the coupled trajectory (not necessarily deterministic). For any test function $\phi \in C_c^\infty(\mathbb{R}^d)$, we have

$$\frac{d}{dt} \mathbb{E}[\varphi(x_t)] = \mathbb{E}[\nabla \varphi(x_t) \cdot \dot{x}_t],$$

by the expectation of the derivative of the test function.

Expanding the expectation into integrals at a fixed t :

$$\begin{aligned} \mathbb{E}[\nabla \varphi(x_t) \cdot \dot{x}_t] &= \iint_{\mathbb{R}^d \times \mathbb{R}^d} \nabla \varphi(x) \cdot \dot{x} \mu_t(dx, d\dot{x}) \\ &= \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot \left(\int \dot{x} p(\dot{x} | x, t) d\dot{x} \right) p_t(x) dx \\ &= \int_{\mathbb{R}^d} \nabla \varphi(x) \cdot v^*(x, t) p_t(x) dx = \mathbb{E}[\nabla \varphi(x_t) \cdot v^*(x, t)]. \end{aligned}$$

where $v^*(x, t) \equiv \mathbb{E}[\dot{x}_t | x_t = x, t] = \int_{\mathbb{R}^d} \dot{x} p(\dot{x} | x, t) d\dot{x}$. Thus the random path induces the weak identity with $v = v^*$.

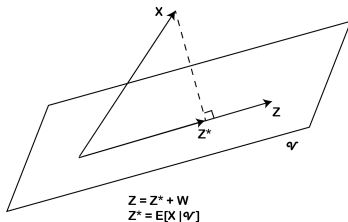
Theoretical Support: Conditional Expectation

Theorem

Let X be an integrable random variable. Then for each σ -algebra $\mathcal{V}$ and $Z \in \mathcal{V}$, $Z^* = \mathbb{E}(X|\mathcal{V})$ solves the least square problem

$$\operatorname{argmin}_{Z \in \mathcal{V}} \|Z - X\|,$$

where $\|Z\| = (\int Z^2 dP)^{\frac{1}{2}}$.



Hence we have $L = \mathbb{E} \|v_\theta(x_t, t) - \dot{x}_t\|^2$.

Practical Training Recipe (Per-Iteration)

- 1 Sample $t \sim w(t)$ (typically uniform on $[0, 1]$).
- 2 Sample $(x_t, \dot{x}_t)$ from the chosen path coupling.
- 3 Regress $\dot{x}_t$ on (x_t, t) via the MSE:

$$\mathcal{L} = \|v_\theta(x_t, t) - \dot{x}_t\|^2.$$

- 4 Update θ by SGD; no density estimation, no reverse simulation is needed during training.

Remark

Different path/coupling choices change v^* and the irreducible variance $\mathbb{E}\|\dot{x}_t - v^*\|^2$, affecting sample efficiency and optimization, but the optimal target remains the conditional mean.

Key Takeaways

- The correct notion of “matching the path” is the weak continuity identity for all test functions.
- The only deterministic field depending on (x, t) that preserves this identity is the conditional mean velocity

$$v^*(x, t) = \mathbb{E}[\dot{x}_t \mid x_t].$$

- The flow-matching MSE is precisely the L^2 projection selecting v^* :

$$v_\theta \xrightarrow[\text{minimize}]{\mathbb{E}\|v_\theta(x_t, t) - \dot{x}_t\|^2} v^*.$$

Outline

- ① Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- ② Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- ③ Applications in AI for Science: From Sequence to structure prediction

Unified Interpretation: Diffusion and Flow Matching

From

- ① Score model $s_\theta(x_t) \rightarrow s_{\theta^*}(x_t) \equiv \mathbb{E}[\nabla \log p(x_t|x_0) | x_t]$
- ② X prediction model (denoising model, x_0 model) $x_\theta(x_t) \rightarrow x_{\theta^*}(x_t) \equiv \mathbb{E}[x_0 | x_t]$;
- ③ ϵ prediction model $\epsilon_\theta(x_t) \rightarrow \epsilon_{\theta^*}(x_t) \equiv \mathbb{E}[\epsilon | x_t]$.

and

$$v^*(x_t) = \mathbb{E}[\dot{x}_t | x_t]$$

, what is the relationship between optimal $v_\theta(x, t)$ and the above models? Hints:

$$v_{\theta^*}(x_t) = \mathbb{E}[\dot{x}_t | x_t] = \mathbb{E}[\dot{\alpha}(t)x_t + \dot{\sigma}(t)\epsilon | x_t]. \quad (59)$$

and

$$s(x_t) = \mathbb{E}_x[\nabla_{x_t} \log p(x_t|x_0)|x_t] = \mathbb{E}_x\left[-\frac{x_t - \alpha x_0}{\sigma^2} | x_t\right] = -\frac{x_t - \alpha \mathbb{E}[x_0|x_t]}{\sigma^2}. \quad (60)$$

Outline

- 1 Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- 2 Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- 3 Applications in AI for Science: From Sequence to structure prediction

Gaussian conditional paths: mathematical derivation

Assumption: Gaussian conditional paths

$$q_t(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_t(\mathbf{x}_0), \sigma_t^2 \mathbf{I}), \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}. \quad (61)$$

Mathematical derivation: Taking the time derivative of the path:

$$\frac{d}{dt} \mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}. \quad (62)$$

Conditional expectation: Since $\boldsymbol{\epsilon}$ is independent of $\mathbf{x}_t$ given $\mathbf{x}_0$, we have:

$$v = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)). \quad (63)$$

Mathematical analysis of path choices

1. Linear bridge (optimal transport):

- $\mu_t(\mathbf{x}_0) = (1 - t) \mathbf{x}_0$, $\sigma_t = t \sigma_{\max}$
- Mathematical property: Minimizes Wasserstein-2 distance
- Velocity: $v = \frac{\sigma_{\max}}{t}(\mathbf{x} - \mathbf{x}_0)$
- Advantage: Direct connection to optimal transport theory

2. VP-like (DDPM-inspired):

- $\mu_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$
- Mathematical property: Preserves energy structure
- Velocity: $v = \dot{\alpha}(t) \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha(t) \mathbf{x}_0)$
- Advantage: Compatible with existing diffusion model schedules

3. VE-like (SMLD-inspired):

- $\mu_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$ increasing
- Mathematical property: Preserves data structure
- Velocity: $v = \frac{\dot{\sigma}(t)}{\sigma(t)}(\mathbf{x} - \mathbf{x}_0)$
- Advantage: Simple form, good for high-dimensional data

Derivation: VE/VP target forms

The interpolation path is given by

$$\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \epsilon, \quad (64)$$

where $\epsilon \sim \mathcal{N}(0, 1)$.

VE-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$. Then

$$v = 0 + \dot{\sigma}_t \epsilon = 0 + \frac{\dot{\sigma}_t}{\sigma_t} (\mathbf{x} - \mathbf{x}_0). \quad (65)$$

VP-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t) \mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$. Using $\mathbf{u}_t = \dot{\boldsymbol{\mu}}_t + (\dot{\sigma}_t/\sigma_t)(\mathbf{x} - \boldsymbol{\mu}_t)$, we get

$$v = \dot{\alpha} \mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} - \frac{\dot{\sigma}_t}{\sigma_t} \alpha \mathbf{x}_0 \quad (66)$$

$$= \frac{\dot{\sigma}_t}{\sigma_t} \mathbf{x} + \left(\dot{\alpha} - \alpha \frac{\dot{\sigma}_t}{\sigma_t} \right) \mathbf{x}_0. \quad (67)$$

Outline

- ① Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- ② Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- ③ Applications in AI for Science: From Sequence to structure prediction

Flow Matching: mathematical training objective

Theoretical foundation: Given conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$, the optimal velocity field satisfies:

$$\mathbf{v}^*(\mathbf{x}, t) = \mathbb{E}[\dot{\mathbf{x}}_t \mid \mathbf{x}_t] \quad (68)$$

Training procedure:

- 1 Sample $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, $\mathbf{x}_t \sim q_t(\cdot \mid \mathbf{x}_0)$
- 2 Compute analytic target $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$ from chosen path
- 3 Minimize regression loss

Mathematical objective:

$$J_{\text{FM}} = \mathbb{E}[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t\|^2]. \quad (69)$$

Key mathematical advantage: No score estimation needed; all targets are analytic!

Computational benefits:

- Stable gradients (no exploding/vanishing)
- Parallelizable across time steps
- No special architectural requirements

Mathematical foundation of inference

Theoretical basis: The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \quad (70)$$

Mathematical properties:

- Existence: Under Lipschitz conditions, solutions exist and are unique
- Consistency: The ODE preserves the continuity equation
- Stability: Deterministic integration avoids accumulation of stochastic errors

Generation process:

- ① Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
- ② Integrate ODE backwards: $\frac{d}{dt} \mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \rightarrow 0$
- ③ Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

Mathematical advantages:

- Deterministic sampling (reproducible)
- Fast with large solver steps
- No stochastic correctors needed

Outline

- ① Understand diffusion models from a mathematical perspective
 - Score Matching: From Explicit Score Matching to Denoising Score Matching
 - Unified View: Score, x_0 , and Epsilon Models
 - Potential Score Matching
 - Diffusion Models: SMLD and DDPM
 - Continuous-Time Formulation
- ② Flow Matching: Learning Probability Flows
 - Probability Path and Coupling
 - Unified Interpretation: Diffusion and Flow Matching
 - Conditional Paths and Velocity Targets
 - Flow Matching Training and Inference
- ③ Applications in AI for Science: From Sequence to structure prediction

SimpleFold: concept & Setup

Goal. Predict full-atom 3D structures from an amino-acid sequence using a general-purpose Transformer trained with flow matching.

Design stance.

- No MSA, no explicit pair representations, no triangular updates, no equivariant blocks.
- Treat sequence as a conditioning signal (like a text prompt) and generate all-atom coordinates.

Flow-Matching Scheme

Continuous-time generation.

$$d\mathbf{x}_t = \mathbf{v}_\theta(\mathbf{x}_t, t) dt, \quad t \in [0, 1]$$

Map a tractable prior p_0 (Gaussian) to data p_D via a learned velocity field.

Training interpolant (rectified flow).

$$\mathbf{x}_t = t\mathbf{x} + (1-t)\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}), \quad \mathbf{x} \sim p_D$$

Target velocity: $\mathbf{v}_t = \mathbf{x} - \boldsymbol{\epsilon}$.

Loss: $\mathbb{E}[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{v}_t\|_2^2]$.

Sampling

Stochastic flow sampling. Initialize $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$, integrate to $t = 1$ with Euler–Maruyama:

$$d\mathbf{x}_t = \mathbf{v}_\theta(\mathbf{x}_t, s, t) dt + \frac{1}{2}w(t) \mathbf{s}_\theta(\mathbf{x}_t, s, t) dt + \sqrt{\tau w(t)} d\bar{\mathbf{W}}_t,$$

where $\mathbf{s}_\theta = \frac{t \mathbf{v}_\theta - \mathbf{x}_t}{1 - t}$ and $w(t) = \frac{2(1 - t)}{t + \eta}$.

Role of τ . Controls diversity/accuracy trade-off for ensembles.

Folding with Flow Matching

Conditioning. Given sequence s and N_a heavy atoms:

$$\mathbf{x}_t, \mathbf{x}, \boldsymbol{\epsilon} \in \mathbb{R}^{N_a \times 3}, \quad \mathbf{v}_\theta = \mathbf{v}_\theta(\mathbf{x}_t, s, t)$$

Main objective (per-atom average).

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\mathbf{x}, s, \boldsymbol{\epsilon}, t} \left[\frac{1}{N_a} \|\mathbf{v}_\theta(\mathbf{x}_t, s, t) - (\mathbf{x} - \boldsymbol{\epsilon})\|_2^2 \right]$$

Structural term (LDDT-inspired). One-step Euler estimate $\hat{\mathbf{x}}(\mathbf{x}_t) = \mathbf{x}_t + (1 - t)\mathbf{v}_\theta(\mathbf{x}_t, s, t)$ and penalize local distance errors:

$$\mathcal{L}_{\text{LDDT}} = \mathbb{E} \left[\frac{\sum_{i \neq j} \mathbf{1}(\delta_{ij} < C) \sigma(\|\delta_{ij} - \hat{\delta}_{ij}^t\|)}{\sum_{i \neq j} \mathbf{1}(\delta_{ij} < C)} \right]$$

Total loss: $\mathcal{L} = \mathcal{L}_{\text{FM}} + \alpha(t)\mathcal{L}_{\text{LDDT}}$.

Timestep resampling. $p(t) = 0.98 \text{LogisticNormal}(0.8, 1.7) + 0.02\mathcal{U}(0, 1)$, biased toward $t \rightarrow 1$ for fine details.

Architecture Overview

Three modules, one block type.

- 1 Atom encoder (local attention): noisy coords $\mathbf{x}_t$ + atom features $\rightarrow$ atom tokens $\mathbf{a} \in \mathbb{R}^{N_a \times d_a}$.
- 2 Residue trunk (heavy): group/pool atom tokens to residue tokens $\mathbf{r} \in \mathbb{R}^{N_r \times d_a}$, concat with frozen PLM embeddings $\mathbf{e} \in \mathbb{R}^{N_r \times d_e}$ from ESM2-3B, process with standard Transformer blocks with adaptive layers.
- 3 Atom decoder (local attention): ungroup residue tokens back to atoms, add skip from encoder, predict velocity $\hat{\mathbf{v}}_t$.

Blocks. QK-normalization, SwiGLU FFN, adaptive scale/shift by t ; all non-equivariant, no pair/triangle stacks.

Positional encodings. RoPE in trunk; 4D axial RoPE in atom modules (3D reference conformer coords + residue index).

Design Choices vs Prior Work

- No MSA, no explicit pair representation, no triangular updates.
- Learn $SO(3)$ symmetries via data augmentation rather than equivariance constraints.
- Hierarchical compute: fine (atoms) $\rightarrow$ coarse (residues) $\rightarrow$ fine (atoms).
- Frozen PLM (ESM2-3B) supplies sequence conditioning; folding model is trained end-to-end with flow matching.

Confidence Module (pLDDT)

What. A separate 4-layer Transformer predicts per-residue pLDDT (0–100).

How.

- Freeze the folding model; feed generated $\hat{\mathbf{x}}$ with $t=1$ to get final residue tokens.
- Train pLDDT head with cross-entropy over 50 bins.

Use. Correlates with LDDT- $C\alpha$ and can score structures from other models.

Training Data & Regime

Data sources.

- PDB (cutoff May 2020) $\sim 160\text{k}$ structures.
- AFDB SwissProt subset (pLDDT mean >85 , std <15) $\sim 270\text{k}$.
- AFESM representatives filtered at pLDDT > 0.8 $\sim 1.9\text{M}$.
- For 3B model: extended AFESM-E up to $\sim 8.6\text{M}$ distilled structures (max 10 per cluster, mean pLDDT > 80).

Training.

- Pre-train at seq length 256; $\alpha(t) = 1$; AdamW $1\text{e}-4$; EMA 0.999; large effective batch.
- Finetune on PDB+SwissProt at seq length 512; $\alpha(t) = 1 + 8 \text{ReLU}(t - 0.5)$ up to 5 near $t=1$.

Structure and Results

Type	Model	TM-score $\uparrow$	GDT-TS $\uparrow$	LDDT $\uparrow$	LDDT-C $_{\alpha}$ $\uparrow$	RMSD $\downarrow$
<i>CAMEO22</i>						
MSA-based	RoseTTAFold (Baek et al., 2021)	0.780 / 0.860	0.715 / 0.775	0.575 / 0.605	0.798 / 0.827	5.721 / 2.864
	AlphaFlow (Jing et al., 2024a)	0.840 / 0.927	0.808 / 0.853	0.741 / 0.798	0.855 / 0.893	3.846 / 2.122
	AlphaFold2 (Jumper et al., 2021)	0.863 / 0.942	0.844 / 0.903	0.816 / 0.856	0.893 / 0.923	3.578 / 1.857
	RoseTTAFold2 (Baek et al., 2023)	0.864 / 0.947	0.845 / 0.904	0.727 / 0.767	0.893 / 0.926	3.571 / 1.707
PLM-based	ESM3 (Hayes et al., 2025)	0.746 / 0.840	0.694 / 0.758	-	-	-
	ESMDiff (Lu et al., 2024a)	0.754 / 0.847	0.701 / 0.760	-	-	-
	EigenFold (Jing et al., 2023)	0.750 / 0.840	0.710 / 0.790	-	-	-
	OmegaFold (Wu et al., 2022)	0.805 / 0.899	0.767 / 0.844	0.746 / 0.815	0.829 / 0.892	5.294 / 2.622
	ESMFlow (Jing et al., 2024a)	0.818 / 0.893	0.774 / 0.832	0.696 / 0.745	0.827 / 0.867	4.528 / 2.693
	ESMFold (Lin et al., 2023b)	0.853 / 0.933	0.826 / 0.875	0.792 / 0.834	0.871 / 0.906	3.973 / 2.019
Ours	SimpleFold-100M	0.803 / 0.878	0.746 / 0.787	0.721 / 0.752	0.822 / 0.852	4.897 / 2.855
	SimpleFold-360M	0.826 / 0.905	0.782 / 0.841	0.773 / 0.803	0.844 / 0.878	4.775 / 2.681
	SimpleFold-700M	0.829 / 0.915	0.788 / 0.845	0.775 / 0.809	0.850 / 0.886	4.557 / 2.423
	SimpleFold-1.1B	0.833 / 0.924	0.793 / 0.851	0.776 / 0.807	0.850 / 0.883	4.350 / 2.334
	SimpleFold-1.6B	0.835 / 0.916	0.799 / 0.864	0.782 / 0.816	0.853 / 0.889	4.397 / 2.187
	SimpleFold-3B	0.837 / 0.916	0.802 / 0.867	0.773 / 0.802	0.852 / 0.884	4.225 / 2.175
<i>CASP14</i>						
MSA-based	RoseTTAFold (Baek et al., 2021)	0.654 / 0.678	0.562 / 0.572	0.464 / 0.456	0.705 / 0.723	9.676 / 6.420
	AlphaFlow (Jing et al., 2024a)	0.740 / 0.812	0.661 / 0.711	0.632 / 0.662	0.767 / 0.799	7.091 / 3.949
	RoseTTAFold2 (Baek et al., 2023)	0.802 / 0.881	0.740 / 0.824	0.638 / 0.669	0.824 / 0.869	6.744 / 3.292
	AlphaFold2 (Jumper et al., 2021)	0.845 / 0.907	0.783 / 0.855	0.778 / 0.817	0.856 / 0.897	5.027 / 3.015
PLM-based	ESMDiff (Lu et al., 2024a)	0.521 / 0.499	0.447 / 0.430	-	-	-
	ESM3 (Hayes et al., 2025)	0.534 / 0.567	0.459 / 0.488	-	-	-
	EigenFold (Jing et al., 2023)	0.590 / 0.637	0.539 / 0.575	-	-	-
	ESMFlow (Jing et al., 2024a)	0.627 / 0.679	0.539 / 0.544	0.525 / 0.539	0.669 / 0.730	10.503 / 6.974
	OmegaFold (Wu et al., 2022)	0.693 / 0.773	0.625 / 0.723	0.627 / 0.726	0.715 / 0.824	9.845 / 4.042
	ESMFold (Lin et al., 2023b)	0.701 / 0.792	0.622 / 0.711	0.637 / 0.705	0.725 / 0.802	8.679 / 4.016
Ours	SimpleFold-100M	0.611 / 0.628	0.513 / 0.544	0.537 / 0.549	0.659 / 0.685	11.157 / 8.976
	SimpleFold-360M	0.674 / 0.758	0.585 / 0.654	0.617 / 0.657	0.703 / 0.762	9.382 / 4.828
	SimpleFold-700M	0.680 / 0.767	0.591 / 0.668	0.630 / 0.674	0.714 / 0.763	9.289 / 4.431
	SimpleFold-1.1B	0.697 / 0.796	0.607 / 0.668	0.640 / 0.676	0.723 / 0.758	9.249 / 4.462
	SimpleFold-1.6B	0.712 / 0.801	0.630 / 0.709	0.660 / 0.699	0.741 / 0.798	8.424 / 4.722
	SimpleFold-3B	0.720 / 0.792	0.639 / 0.703	0.666 / 0.709	0.747 / 0.829	7.732 / 3.923

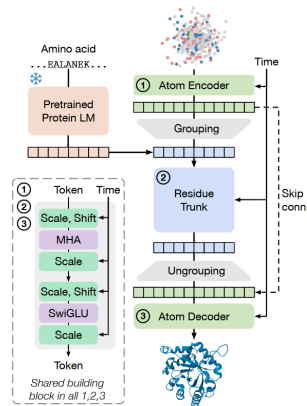


Figure 2 Overview of SimpleFold's architecture built on general-purpose standard Transformer block with adaptive layers. Atom encoder, residue trunk, and atom decoder all share the same general-purpose building block. Our model circumvents the need for pair representations or triangular updates.

Results

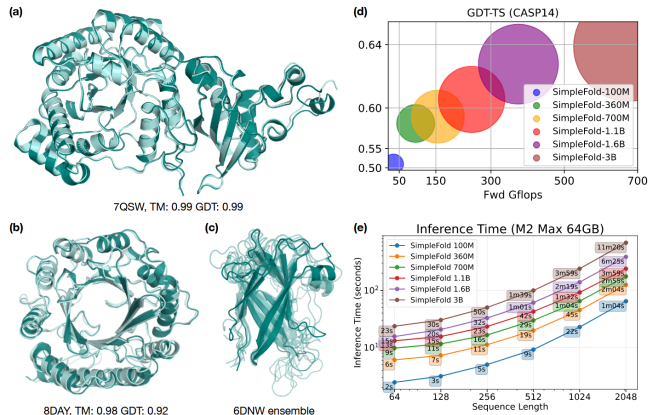


Figure 1 Example predictions of SimpleFold on targets (a) chain A of 7QSW (RubisCO large subunit) and (b) chain A of 8DAY (Dimethylallyltryptophan synthase 1), with ground truth shown in light aqua and prediction in deep teal. (c) Generated ensembles of target chain B of 6NDW (Flagellar hook protein FlgE) with SimpleFold finetuned on MD ensemble data. (d) Performance of SimpleFold on CASP14 with increasing model sizes from 100M to 3B. (e) Inference time of different sizes of SimpleFold on consumer level hardware, i.e., M2 Max 64GB Macbook Pro.

Scalability

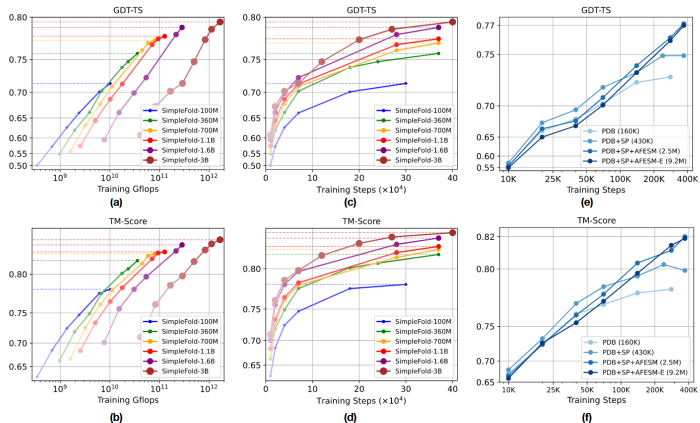


Figure 4 Scaling behavior of SimpleFold. Training Gflops vs. folding performance on GDT-TS and (b) TM-score. Training steps vs. folding performance on (c) GDT-TS and (d) TM-score. How data scale affects the performance (e) GDT-TS and (f) TM-score. All models are benchmarked on CAMEO22.

The Future of AI for Science

- Democratization: Making advanced scientific tools accessible to all researchers
- Interdisciplinary collaboration: Breaking down silos between scientific fields
- Scientific method evolution: How AI changes the scientific process
- Grand challenges: Tackling humanity's biggest scientific problems

Welcome inputs

Any comments?

Welcome your inputs!